

Towards Temporal Reasoning: A Survey of Concepts and Benchmarks for Temporal GNN/Graph-LLM Hybrids

Josefin Kelber, Dániel Sajben, Jiyun Gong, Mengfan Wu

Abstract

Large Language Models (LLMs) excel at natural language understanding, but struggle with complex temporal reasoning and dynamic knowledge evolution. Conversely, Graph Neural Networks (GNNs) provide robust relational reasoning yet typically operate on static snapshots, failing to capture the fluid dynamics of real-world systems. To bridge this gap, Temporal GNN-LLM Hybrids (TGLH) have emerged as an integrated paradigm combining semantic processing with structural evolution modeling. While prior surveys have addressed GNN-LLM hybrids, temporal graphs, and temporal GNNs independently, to the best of our knowledge, no work has systematically analyzed their intersection. This survey provides the first taxonomy specifically dedicated to TGLH, categorizing architectures into LLM-centered, GNN-centered, and fusion-based approaches and subsequently by their conceptual main ideas. We formalize TGLHs by defining them as models that learn mappings from temporal input sequences and dynamic graphs to outputs, explicitly distinguishing them from static hybrids. Furthermore, we synthesize the evaluation landscape across three core task categories — temporal question answering, forecasting and completion, and graph generation — identifying metrics for evaluating the limitations of current systems. By consolidating recent advances and highlighting open challenges such as long-range dependency modeling, entity evolution tracking, and recency-context trade-offs, this survey establishes a foundation for future research into AI systems capable of robust temporal reasoning.

1 Introduction

In recent years, the scaling of *Large Language Models (LLMs)* has enabled their application to a wide range of domains and tasks [64]. However, LLMs are known to struggle with issues such as hallucinations and complex multi-hop reasoning [70]. In contrast, *Graph Neural Networks (GNNs)* are specifically designed to operate on graph-structured data, offering complementary strengths in relational reasoning and representation learning [96]. At the same time, GNNs are not designed as general-purpose language understanding models: when text is involved, they typically rely on additional graph construction and/or separate text encoders rather than natively handling long, unstructured documents [84]. These complementary strengths and weaknesses have

led to a growing interest in hybrid architectures that integrate GNNs and LLMs [45].

Nevertheless, most existing GNN-LLM hybrids are built upon static graph snapshots, which limits their ability to model temporal dynamics such as event ordering, duration, and temporal evolution [55]. Consequently, these models often fail to perform robust temporal reasoning or to capture dynamically changing graph structures. This limitation hinders their practical applicability, given that many real-world systems are inherently dynamic. These range from general domains like social networks, transportation systems, and evolving narratives [27] to specialized, high-stakes environments like stock market volume prediction in finance [92] or multi-modal electronic health records in healthcare [106].

To address this gap, this paper surveys recent approaches that incorporate temporal aspects into GNN-LLM hybrids, which we label **Temporal GNN-LLM Hybrids (TGLHs)**. We categorize these approaches into three groups: LLM-centered, GNN-centered, and fusion-based architectures. For each category, we highlight the main paradigms for integration, such as using GNN-generated graph-aware embeddings for LLMs, or integrating GNNs and LLMs at the training level (see Fig. 2). We additionally include LLM-graph hybrids in our taxonomy where appropriate. Furthermore, we survey benchmarks and metrics that can be used to evaluate TGLHs and sort them by task categories (see Tab. 1). Consequently, our key contributions are:

1. A structured overview of recent Temporal GNN-LLM Hybrid architectures.
2. A summary of relevant benchmarks for evaluating such models.

The remainder of this paper is organized as follows. Section 2 provides necessary background on graphs, GNNs, LLMs, and existing GNN-LLM hybrids. Section 3 formalizes the concept of Temporal GNN-LLM Hybrids. Section 4 discusses the different emerging TGLH architectures in detail, while Section 5 provides an overview of suitable benchmarks. In Section 6 we highlight the performance of TGLHs as well as their limitations regarding costs and scalability. Finally, Section 7 discusses the findings and outlines possible directions for future research.

2 Background

In this section, we briefly repeat the general concepts of Large Language Models (see 2.1), graphs (see 2.2) and Graph

Neural Networks (see 2.3). Due to the relevance for our topic, we specifically expand on retrieval augmented generation, temporal graphs, and dynamic Graph Neural Networks. Furthermore, we summarize the current hybrids of Graph Neural Networks and Large Language Models (see 2.4).

2.1 Large Language Models (LLMs)

LLMs are neural networks that learn complex linguistic patterns from massive text corpora, enabling them to capture long-range dependencies, model contextual relationships, and produce coherent text [64]. LLMs can be specialized for domain-specific tasks through fine-tuning and parameter-efficient techniques [24, 61]. Most state-of-the-art LLMs are based on the Transformer architecture [76], which utilizes the self-attention mechanism [5]:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V,$$

with queries Q , keys K , values V , and dimensionality d_k . Self-attention treats an input sequence as a fully connected graph where each token attends to all others. However, its quadratic complexity imposes practical limitations on context length. Furthermore, LLMs struggle with multi-hop reasoning [6, 94] and hallucinations [28, 69]. A primary cause of hallucination is temporal misalignment: the knowledge encoded within an LLM’s parameters is static, effectively frozen at the time of training [44, 78].

Graph Retrieval-augmented Generation (Graph RAG). Retrieval-augmented Generation (RAG) [42] address this static knowledge limitation, by allowing LLMs to query up-to-date external sources. While standard RAG typically retrieves isolated document chunks based on vector similarity [91], this approach often fails to capture complex structural dependencies. *GraphRAG* bridges this gap by utilizing structured *knowledge graphs* (KGs) as the retrieval source. Unlike text-based retrieval, GraphRAG can retrieve multi-hop relational paths or subgraphs, enabling the model to trace explicit connections between entities [60]. For instance, in biomedical applications, a model might traverse a KG to identify entities linked by complex relationships—such as specific protein targets—before synthesizing the retrieved paths, thereby providing transparent reasoning that pure semantic similarity cannot achieve. Despite these advances, both standard and Graph RAG are fundamentally constrained by the LLM’s finite context length and the empirically observed “lost-in-the-middle” phenomenon, where models struggle to attend to information positioned in the middle of long contexts [50].

2.2 Graphs

A graph is defined as $G = (V, E)$, where V is the set of nodes and E is the set of edges [104]. An edge is denoted as $e_{ij} = (v_i, v_j)$ representing a directed edge from v_j to v_i . The neighborhood of a node v is $\mathcal{N}(v) = \{u \in V \mid (v, u) \in E\}$.

The adjacency matrix $A \in \{0, 1\}^{n \times n}$ is defined as

$$A_{ij} = \begin{cases} 1, & \text{if } (v_i, v_j) \in E, \\ 0, & \text{otherwise.} \end{cases}$$

Nodes may have feature vectors $X \in \mathbb{R}^{n \times d}$ with $x_v \in \mathbb{R}^d$, and edges may have feature vectors $X_e \in \mathbb{R}^{m \times c}$ with $x_{e(v,u)} \in \mathbb{R}^c$ [15]. A graph is *directed* if all edges have directions. An *undirected* graph is a special case in which each connection is represented by a pair of inverse-direction edges.

Content-associated Heterogeneous Graph (C-HetG). A *content-associated heterogeneous graph* (C-HetG) is defined as $G = (V, E, O_V, R_E)$, where O_V is the set of node types, R_E the set of relation types, and $|O_V| + |R_E| \geq 3$ [98]. Nodes contain heterogeneous content such as attributes, text, or images. Multimodal attributes often model the underlying data better and therefore improve contextual understanding, but aggravate unified attribute processing [44].

Temporal Graphs. In a *static graph* $G = (V, E)$, the vertices $V = \{v_1, v_2, \dots, v_{|V|}\}$ and the edges $E \subseteq V \times V$ are fixed, i.e., they do not change over time [35]. Informally, *dynamic graphs* (also called temporal, evolving, or time-varying) are graphs that change over time [54] in topology (i.e., V, E) and/or attributes X . *Spatial-temporal graphs* $G(t) = (V, E, X(t))$, are special cases where node features change over time, while the topology is fixed [38].

In general, we can distinguish between modeling time in a discrete or continuous way. A *continuous-time dynamic graph* (CTDG) is a pair (G, O) . The static graph G is the initial state of a dynamic graph at time t_0 . The set O consists of observations, i.e., tuples of the form (event type, event, timestamp). Event types include, for example, edge addition, edge deletion, node addition, node deletion, node splitting, and node merging. By updating G according to observations O that occurred before a time point t , we can produce a static graph, called a snapshot [35]. Alternatively, CTDGs are often formalized as a stream of timestamped events, such as edge interactions (u, v, t) , allowing for fine-grained modeling of asynchronous updates [29]. Under the assumption that time is discrete, a dynamic graph is a series of snapshots, i.e., a sequence of static graphs $\{G^1, G^2, \dots, G^T\}$ sampled at regularly spaced times. $G^t = \{V^t, E^t\}$ is the graph at snapshot t . V^t is the vertex set in G^t , and E^t is the edge set in G^t . While working with *discrete-time dynamic graphs* (DTDGs) is often less complex than continuous time modeling, it can lead to information loss [35].

Temporal Knowledge Graphs (TKGs). A KG provides structured real-world knowledge in the form of facts (often represented through subject-predicate-object triples): Two nodes (i.e., entities) are connected by edges that represent the relations between them. Compared to general graphs, KGs are explicitly multi-relational, i.e., they have typed edges with different semantics and are typically paired

with an ontology schema [23]. Although large parts of the literature assumed static facts, there are approaches which consider the temporal dynamics of facts. Unlike general dynamic graphs, which focus on the topological evolution of the network (e.g., node additions), *Temporal KGs (TKGs)* specifically model the temporal validity of semantic facts – e.g., $\langle \text{Obama, PresidentOf, USA, [2009, 2017]} \rangle$. For a detailed introduction to temporal KGs, we refer to Cai et al. [8], who additionally categorize approaches to solving the temporal reasoning task of *Temporal Knowledge Graph Completion (TKGC)* into six categories: time-included tensor decomposition, time-based transformation, dynamic embedding, learning from knowledge graph snapshot, reasoning with historical context, and temporal logical rules.

2.3 Graph Neural Networks (GNNs)

A GNN computes node representations via a neighborhood aggregation scheme, often termed *Message Passing*. Given a graph $G = (V, E)$ and a node $n \in V$, a GNN updates embeddings layer-wise by first aggregating the representations of its neighbors and then combining this aggregated message with the node’s current state. Formally, the update at layer k is defined as:

$$a_n^{(k)} = \text{AGGREGATE}^{(k)}\left(\left\{h_u^{(k-1)} : u \in \mathcal{N}(n)\right\}\right),$$

$$h_n^{(k)} = \text{COMBINE}^{(k)}\left(h_n^{(k-1)}, a_n^{(k)}\right),$$

with initialization $h_n^{(0)} = x_n$, where x_n denotes initial node features [90]. Popular architectures like *Graph Convolutional Networks (GCN)* [37] and *GraphSAGE* [21] utilize this paradigm to capture local structural patterns.

However, this standard formulation is inherently *snapshot-based*. It maps a single graph state to embeddings, treating the adjacency matrix A as fixed. Consequently, standard GNNs cannot naturally represent temporal order, recency, or evolutionary patterns (e.g., distinguishing an edge formed minutes ago from one formed years ago). Applying these static models to dynamic systems requires treating the graph as independent snapshots, which discards crucial sequential dependencies across time.

Attention-Inspired GNNs. To improve the aggregation capability, mechanisms inspired by the attention mechanism have been adapted for graphs. *Graph Attention Networks (GAT)* [77] introduce an attention mechanism to assign learnable weights to neighbors, allowing the model to prioritize more relevant connections rather than aggregating all neighbors equally. While these architectures share the underlying attention mechanism with LLMs—facilitating architectural alignment in hybrids—they remain fundamentally limited to static structural processing unless explicitly adapted for time.

Temporal Graph Neural Networks (TGNNs). To address the dynamic nature of real-world data, *Temporal GNNs*

(TGNNs) — also called *Dynamic GNNs (DGNNs)* — extend the message-passing paradigm to evolve over time. These approaches generally fall into two categories:

1. **Discrete-Time Models (Snapshot-based):** Discrete-time models process a sequence of graph snapshots $\{G^1, \dots, G^T\}$. Architectures like *EvolveGCN* [59] utilize *Recurrent Neural Networks (RNNs)* to evolve the GNN parameters (weights) or node embeddings from one snapshot to the next. While effective for stable periodic data, they suffer from information loss due to coarse temporal discretization.
2. **Continuous-Time Models (Event-based):** Approaches such as *Temporal Graph Networks (TGN)* [67] and *Temporal Graph Attention Networks (TGAT)* [88] process time as a continuous stream of events. They typically employ a memory module to store the state of each node and update it asynchronously upon every interaction using time-encoding vectors.

Despite their ability to model structural evolution, TGNNs face significant limitations that motivate the need for LLM hybridization. First, they struggle with long-term dependencies due to the “over-smoothing” and “over-squashing” phenomena inherent in recursive aggregation [85]. Second, and most critically, they lack semantic reasoning capabilities. A TGNN can predict *that* a link will occur based on historical frequency, but it cannot explain *why* based on the complex textual context of the interaction (e.g., understanding that a “resignation letter” event implies a future “hiring” event). This semantic gap hinders explainable temporal reasoning, necessitating the integration with LLMs.

2.4 GNN-LLM Hybrids

The idea of combining LLMs and GNNs is to forge a unified architecture that overcomes their individual limitations. LLMs are capable of capturing complex semantics and performing in-context learning and reasoning, yet they often struggle with multi-hop reasoning and factual hallucination. Conversely, GNNs excel at graph-based, well-grounded reasoning by detecting intricate patterns of connections within structured data, but they are often constrained by simplistic initial node features and underutilize the rich semantic information in associated text. Interestingly, the attention mechanism used in Transformer-based LLMs can even be interpreted as a form of message passing across a fully connected graph of tokens, revealing a deep structural parallel between the two approaches [63]. This complementarity suggests that an integrated approach can create models that holistically understand both structure and semantics. Recent studies show that integrating graphs as well as GNNs and LLMs can be beneficial [45]. Approaches of combining GNNs and LLMs can be broadly grouped into three main categories: leveraging LLMs to enhance GNNs, employing

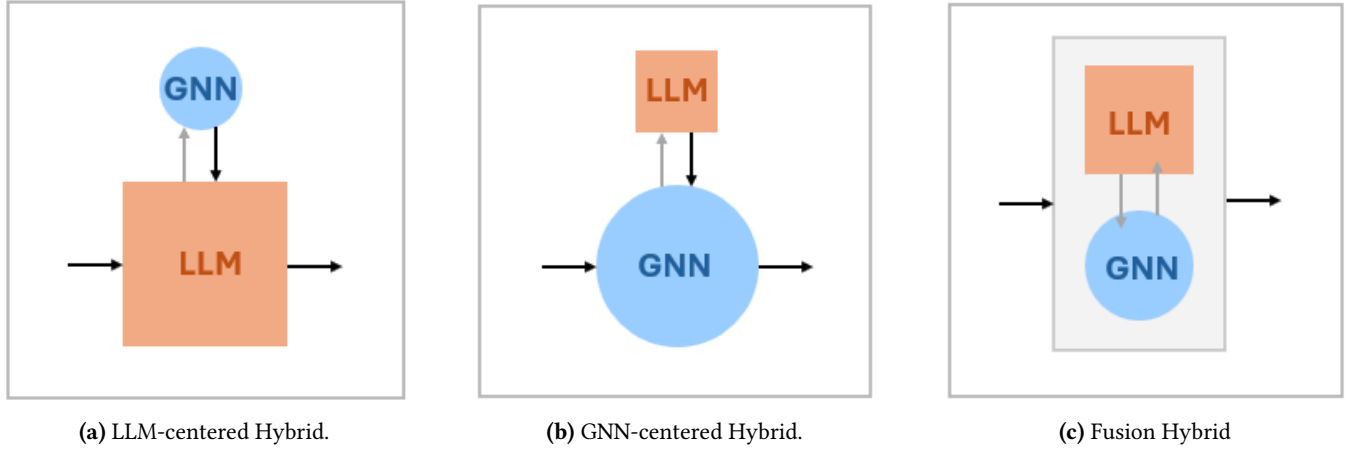


Figure 1. Three paradigms for integrating GNNs and LLMs. LLM-centered hybrids utilize GNNs (or graphs) for improving the LLM’s performance. GNN-centered hybrids utilize LLMs for enhancing graph-based models. Fusion hybrids can be further subdivided into unified and collaborative approaches: unified models integrate GNNs and LLMs into a single architecture, whereas collaborative approaches retain separate components that mutually enhance each other.

GNNs to augment LLMs, and developing unified hybrids that synergistically combine both paradigms (see Fig. 1).

2.4.1 GNN-Enhanced LLMs (GNNs for LLMs). This paradigm focuses on converting graph data into tokens suitable for LLM processing, enabling models to reason over structured relational information. Common techniques include linearizing knowledge graphs into sequential triples, encoding subgraph structures as special tokens, and incorporating graph-derived embeddings into the input space of LLMs. These approaches facilitate KG/Graph-RAG grounding, graph query planning/tool-use, and graph-aware prompting/masking to make generations factual and consistent. Integrating KGs in particular has proven effective, culminating in advanced GraphRAG methods that retrieve structured, relational facts to improve reasoning and mitigate hallucinations [58, 97]. However, even recent and sophisticated approaches often treat the graph as a static snapshot. For instance, techniques that encode graph structures into special tokens for the LLM or directly inject KG triples into the model’s layers still operate on a fixed representation of the graph [18, 101]. These methods lack an intrinsic mechanism for reasoning about how the graph’s structure and attributes evolve over time [29, 44].

2.4.2 LLM-Enhanced GNNs (LLMs for GNNs). One promising approach to integrating LLMs with GNNs is to leverage the LLM as an enhancer of the graph model. In this setup, the GNN remains the primary architecture for reasoning over graph structures, while the LLM supplies rich semantic information to augment the GNN’s input features or training process. This paradigm, termed LLMs-as-Enhancers, is particularly effective when dealing with *text-attributed graphs* (TAGs), where nodes or edges are associated with textual

descriptions. For instance, an LLM—or a smaller language model like BERT or Sentence-BERT—can convert raw text attributes into high-quality vector embeddings. These embeddings then serve as initial node features, enabling the GNN to operate on semantically rich representations from the very beginning. This integration can be realized through feature-level enhancement, where embedding-visible LLMs (such as Sentence-BERT or E5) encode text attributes into node embeddings via a cascading structure. This approach has proven both effective and efficient [13], especially when combined with deep sentence embedding models, which offer strong performance without the need for fine-tuning.

2.4.3 Fusion Hybrids. The pursuit of deep integration between GNNs and LLMs leads to the paradigm of *Fusion Hybrids*. Rather than treating the two as separate components for simple concatenation, this paradigm aims to construct cohesive systems capable of simultaneous and collaborative processing of graph structure and textual semantics. The core philosophy is to facilitate bidirectional information flow, breaking down the barriers between “graph encoding” and “text encoding”.

One prominent path to this unification is through *unified model architectures*. These are end-to-end frameworks natively designed to intertwine graph and text processing. For example, Yang et al. [93] proposed GL-Fusion, which integrates structure-aware attention and graph-text cross-attention within a single model, achieving synergistic encoding of both information types [93].

Another effective path is the *student-teacher collaborative paradigm*, where the LLM and GNN remain as distinct modules but engage in a bidirectional, iterative enhancement loop during training. This approach leverages techniques like knowledge distillation and representation alignment to

allow each model to compensate for the other’s weaknesses – LLMs impart semantic reasoning to GNNs, while GNNs impart structural grounding to LLMs [46, 89].

In the LLM-to-GNN direction, a powerful LLM (teacher) transfers its semantic knowledge to a smaller GNN (student). Research has been done to transfer LLMs’ semantic or reasoning capabilities to GNNs via feature alignment [26], rationale encoding [22], and intermediate model distillation [57], aiming to endow GNNs with enhanced understanding and inference abilities without relying on the LLMs’ direct, resource-intensive text processing.

Conversely, in the GNN-to-LLM direction, graph structures are used to guide and constrain LLMs for improved factual or structural accuracy. For instance, Liu et al. [46] proposed a GNN-guided retrieval-augmented generation framework that preserves coherent multi-hop reasoning paths in a knowledge graph to steer LLM generation. Similarly, Wang et al. [79] aligned GNN node embeddings with an LLM’s token space, enabling zero-shot prediction on graph tasks via these “graph token embeddings”.

These two approaches – unified architectures and the student-teacher paradigm – demonstrate that deep GNN-LLM integration can be achieved either through architectural unity or through tight collaborative training, both moving beyond simple, one-way enhancement.

3 Concept and Definition of Temporal GNN-LLM Hybrids

Although the integration of GNNs and LLMs helps bridge the gap between structural and semantic reasoning [45], the majority of existing hybrids operate under a static assumption: they view the world as a fixed snapshot. This usually does not consider temporal dynamics, such as order in time, duration, and shifts over time. However, many real-world use cases – e.g., modeling narratives, social interactions, or physical systems – are inherently dynamic. Under the term of *Temporal GNN/Graph-LLM Hybrids* (TGLHs), we consider all GNN-LLM hybrid architectures that explicitly consider the dynamic nature of data and model their temporal aspects. Additionally, we include architectures that are hybrids of LLMs and temporal graphs, as temporal graphs can be processed and modified by temporal GNNs as well.

3.1 Formal Definition

We define a Temporal GNN-LLM Hybrid as a hybrid neural architecture that integrates the semantic processing capabilities of a Large Language Model with the structural evolution modeling of a dynamic graph framework. Formally, given a sequence of inputs $X = \{x_1, x_2, \dots, x_t\}$ and a dynamic graph $G(t) = (V_t, E_t)$, a TGLH learns a mapping function $f : (X, G(t)) \rightarrow Y$. We use the term “GNN” in the acronym TGLH broadly to refer to any dedicated module for modeling

temporal graph structure. This includes explicit Graph Neural Networks (e.g., TGN, TGAT), temporal graph encoders, and structure-aware serialization techniques where the dynamic framework is implicit (e.g., serialized history treated as a temporal sequence of edges). Unlike static hybrids where G is invariant, in a TGLH time t is explicitly modeled by one of its components.

3.2 Motivation: Temporal Reasoning

Temporal reasoning generally refers to the ability to model, infer, and reason about events, relations, and structures that evolve over time, including their ordering, duration, and temporal dependencies [2]. The necessity for TGLHs arises from the orthogonal limitations of its constituents regarding time. LLMs suffer from temporal misalignment and factual drift; their parametric knowledge is frozen at the training cutoff [78]. Furthermore, while LLMs can process sequential tokens, they struggle with global temporal reasoning. For instance, answering “What was the company’s stock price one year after the CEO changed?” requires an LLM’s semantic understanding of “CEO change” and a temporal graph’s structural model of the company’s historical relations to pinpoint the correct timestamp and retrieve the fact. Pure LLMs often fail to deduce the state of a system after such a complex series of interleaved events [100]. In contrast, (dynamic) GNNs excel at extrapolating future links based on structural history, but lack the semantic capacity to interpret why an interaction occurred [29].

3.3 Distinction from Related Concepts

To underline the necessity of TGLHs, we highlight the limitations of adjacent research, such as Graph RAG, temporal KGs, time-aware LLMs and dynamic GNNs.

Graph RAG. Standard GraphRAG approaches (see section 2.1) retrieve context from a static graph index. Even if the underlying facts contain dates, semantic similarity dominates over temporal intent. For example, “What did Barack Obama do before 2008?” and “What did Barack Obama do after 2008?” yield nearly identical embeddings [73]. Furthermore, simple passage concatenation of retrieved content can lead to fragmented and inconsistent narratives [95]. Additionally, single-pass retrieval and hallucinations (caused by the inability to distinguish up-to-date information from multiple sources) are inherent limitations of most current GraphRAG approaches [95].

Temporal Knowledge Graphs (TKGs). In TKGs (see section 2.2) edges are annotated with timestamps to limit the time frame in which a relation exists. While this data structure enables modeling of temporal dependencies, TKGs have limited expressiveness for temporal reasoning: Since TKGs are relation-centric, they cannot represent how entities themselves change over time, only how their pairwise relations

do. Sun et al. [73] additionally criticize, that TKGs store abstracted entity–relation triples without linking to the original source.

Time-Aware LLMs. Time-aware language models address temporal drift by conditioning generation on explicit time signals, for example, by prepending timestamps to the input or by biasing attention toward temporally proximal tokens [17, 66]. While this mitigates temporal averaging and improves calibration under temporal distribution shift, such models remain fundamentally sequence-based: time is treated as an auxiliary textual or positional feature rather than as a structured organizing principle. They can not explicitly model evolving relational structures, such as temporal neighborhoods, interaction histories, or dependencies between entities and events over time.

Temporal Graph Neural Networks. TGNNs (see section 2.3) can model evolving graphs, either in discrete time via graph snapshots or in continuous time via event-driven updates. While effective at capturing short-range temporal dependencies, TGNNs inherit fundamental limitations of message-passing architectures, including difficulties with long-term dependency modeling, over-smoothing, and over-squashing when reasoning over deeper graphs and longer temporal spans [3, 56]. Moreover, TGNNs operate primarily on low-dimensional latent node representations and lack the semantic and linguistic understanding required to process unstructured, text-heavy inputs. TGNNs also offer limited support for human-interpretable reasoning, since their predictions are typically expressed in terms of latent embeddings rather than explicit language-level rationales.

4 Temporal GNN-LLM Hybrids Architectures

Analogous to general GNN–LLM hybrids, we categorize temporal GNN–LLM hybrids into LLM-centric, GNN-centric, and collaborative or unified architectures. Moreover, we also include hybrids that combine LLMs with temporal knowledge graphs. Although these approaches are not strictly GNN–LLM hybrids, we consider them as TGLHs, since temporal knowledge graphs can be used as a graph-structured interface for integrating GNN-based architectures. We visualize the taxonomy in figure 2.

4.1 LLM-Centered Architectures

We consider an architecture as LLM-centered TGLH, if the LLM is the main predictor and graphs or GNNs are mainly used as inputs or for retrieval.

4.1.1 Temporal Knowledge Graph RAG. One of the most explored approach for integrating temporal aspects into LLMs is using temporal graphs as retrieval structure and then integrating the retrieved information in the LLM’s

context. In the following we highlight relevant concepts and ideas.

LLM-Generated Event-Centric Graphs. Standard RAG often chunks text at sentence, paragraph or document level. In contrast, temporal GraphRAG the retrieved objects are events and the event-centric graphs are often constructed through utilizing LLMs. For example, *EventRAG* [95] uses an LLM to extract information from text to create an *Event Knowledge Graph (EKG)*, in which events have temporal markers as attributes, and temporal relationships between events are presented by edges between them. *Dynamic Graph RAG (DyG-RAG)* [73] instead creates a dynamic event graph, which has *Dynamic Event Units (DEUs)* as nodes and edges representing the relevance between two DEUs, based on entity co-occurrence and temporal proximity. These DEUs, which are encoded into a dense vector representation and assigned to sentences, are extracted by an LLM, which identifies the temporal expressions within overlapping chunks and assigns them as timestamps to sentences. In contrast, *Temporal GraphRAG (T-GRAG)* [43] builds a TKG of entities and relations, where all nodes and edges are annotated with temporal attributes. It is generated by splitting the corpus into time periods, chunking into text blocks, using an LLM to extract entities and their relationships, and finally attaching timestamps and updating “latest knowledge” over time.

Time-aware Embeddings for Similarity Search. *TimeR⁴* [62] conducts time-aware retrieval from a *Temporal Knowledge Store (TKS)*, i.e., a vector index of TKG quadruples whose embeddings are produced by a time-aware encoder. Candidate facts are retrieved by cosine similarity to the (rewritten) question embedding. To make the encoder sensitive to temporal constraints (not just semantic overlap), *TimeR⁴* fine-tunes the encoder with a contrastive time-aware objective that constructs negative types by corrupting time, content (entities/reasons), or both. *DyG-RAG* [73] uses a dense vector representation that combines the semantic sentence embedding generated by a pretrained encoder and temporal information — through a Fourier time encoder that preserves relative time distances by converting timestamps into a smooth periodic representation.

Iterative Reasoning Paths. Another important point are the traces that the retrievers produce. They can improve the explainability of architectures and enable self-correction. *EventRAG* [95] validates the intermediate reasoning paths that their agent creates iteration-wise against the graph structure of their EKG and therefore enables self-correction and updating of the agent’s goals. *DyG-RAG* [73] converts its retrieved paths — gained by weighted, random walks — into a structured, chronologically ordered event timeline, which is later used for prompting.

Specific Prompting Techniques. For the generation of the final answer, many approaches use specifically constructed

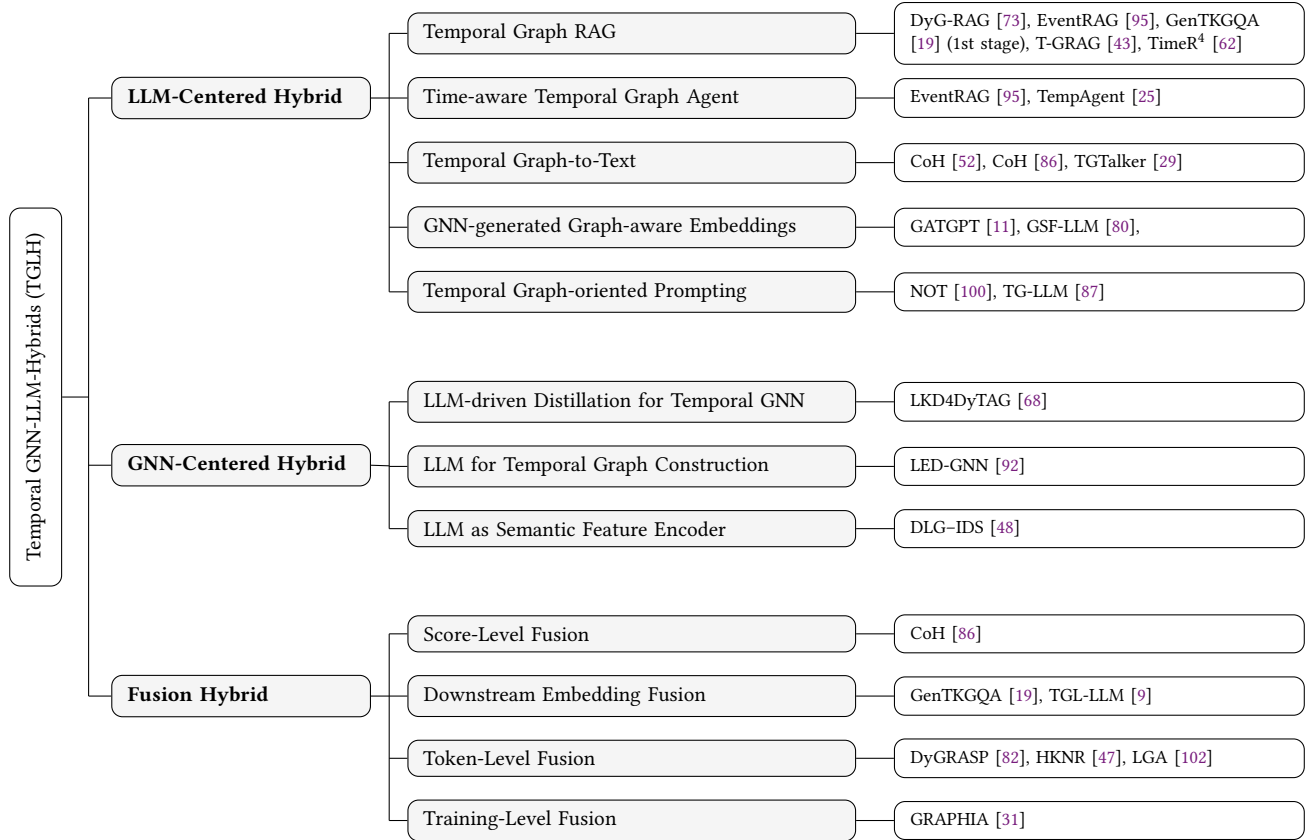


Figure 2. Taxonomy of Temporal GNN-LLM Hybrids (TGLHs), categorized by their primary integration paradigms.

prompting templates. For example, Sun et al. [73] introduces *Time Chain-of-Thought (Time-CoT)*, which uses a structured event timeline that explicitly encodes temporal relations (e.g., intervals, containment, and causal succession) and a temporal reasoning template to prompt LLMs to do temporal verification and reasoning before generating a final answer.

4.1.2 Temporal Graph-to-Text Serializers. While GraphRAG approaches use the graph as retrieval index (and produces walks, subgraphs, or evidence), serialization instead injects a selected part of a temporal graph directly into the LLM prompt by converting it into text. A recent example is *TGTalker* [29], which formulates temporal link prediction as a text-in/text-out task. The LLM — instructed to be a *Temporal Graph Learning (TGL)* agent — has to predict the destination node after receiving a prompt in the form of a set of edges (filtered by recency bias), few-shot link-prediction Q/A pairs, and the query edge. Furthermore, the source node’s most recent neighbors are listed through temporal neighbor sampling. Nodes are assigned as IDs based on their chronological sequence of appearance and edges are represented with parentheses as (src node ID, dst node ID, timestamp), with integer-valued timestamps. *Chain of History (CoH)* by Xia et al. [86], formats TKG facts

(subject, relation, object, timestamp) into fluent sentences with abstract time references – e.g., (Germany, Sign agreement, Denmark, 2023-06-02) is converted to (Germany Sign agreement with Denmark on the 153rd day). The history chain is then used by the LLM as context for TKG prediction. *Chain-of-History (CoH)* by Luo et al. [52] instead learns to predict from serialized histories by fine-tuning the LLM on these constructed sequences. Beyond query-relevant historical facts, it augments the serialized history with neighborhood information around the involved entities/relations, so the LLM can exploit local structural cues from text — this is conceptually similar to TGTalker’s temporal-neighbor injection, but uses a fine-tuned, generative predictor rather than prompt-only link prediction.

4.1.3 Time-aware Graph Agents. Similar to RAG, time-aware graph agents retrieve information from a graph. In fact, the already mentioned EventRAG [95] can be seen as an agentic approach as well. It also use multi-pass retrieval. However, *TempAgent* [25] uses a more complex Temporal Retrieval Toolbox (that has functions such as SearchOnDay, SearchBeforeMonth, and SearchAfterYear) and therefore

enables fine-grained filtering across different time granularities. The agent operates through iterative “Thought $\rightarrow$ Action $\rightarrow$ Observation” cycles. It is guided by custom prompts tailored to specific temporal question types, allowing it to decompose a complex query, select the appropriate temporal tool, and refine its search based on the results.

4.1.4 Graph-Oriented Prompting. While the previous hybrids also used specific prompt templates, they were supplementary and not the core mechanism. This differs for *Narrative-of-Thought (NOT)* [100], which utilizes LLMs for *Temporal Graph Generation (TGG)*, by formatting TGG as a code completion task. NOT prompts the LLM to generate a temporally grounded narrative (i.e., a natural language equivalent of an event graph) before asking it to complete the TGG code. To improve the performance, NOT also includes reference narratives as few-shot demonstrations. While *TG-LLM* [87] additionally uses *supervised fine-tuning (SFT)* to train LLMs to predict aligned timelines (enhanced by CoT bootstrapping and graph data augmentation), it is still heavily prompt-centered: It prompts an LLM to translate the context into a temporal graph and then generates an answer by reasoning on it.

4.1.5 GNN-Generated Graph Embeddings. Instead of serializing graphs into raw text, these hybrids first encode structure (graph topology and/or spatio-temporal dependencies) into continuous embeddings, and then inject those embeddings into an LLM (e.g., as a token sequence, through a projector, or by making attention graph-aware). *GATGPT* [11] improves spatio-temporal imputation by attaching a graph attention module to a GPT-2 backbone. It maps multivariate time series into token embeddings and adds positional embeddings to preserve chronological order. Then the graph attention module aggregates information from neighboring nodes into node representations, and these graph-enriched embeddings become the input sequence to the LLM. During finetuning, the GPT-2 blocks are largely frozen. Similarly, *GSF-LLM* [80] uses a *partially frozen graph attention (PFGA)* mechanism. It first produces token, temporal, and spatial embeddings and fuses them via multi-head cross-attention. The fused representation is then processed by a partially frozen GPT-2 backbone. Later layers are unfrozen, augmented with graph-based attention and finetuned with *Low-Rank Adaptation (LoRA)* [24].

4.2 GNN-Centered Architectures

We call architectures GNN-centered, if a GNN is the primary inference engine, i.e., a GNN consumes an evolving graph and produces the final prediction. A LLM has a supporting role such as provide additional information or supervision.

4.2.1 LLM-driven Distillation for Temporal GNN. One approach is to use a LLM as a teacher, who supervises a temporal GNN. *LLM-driven Knowledge Distillation for Dynamic*

Text-Attributed Graphs (LKD4DyTAG) [68] utilizes *Dynamic Text-Attributed Graphs (DyTAGs)*, which contain temporal and structural information as dynamic graphs do, but additionally include textual information for nodes and edges. They use the teacher LLM to generate textual edge embeddings by processing text descriptions of the edge and the 1-hop neighborhoods of its endpoint nodes. In parallel, the student GNN computes a spatio-temporal edge embedding from non-raw-textual graph signals: node and relation text features (computed by a frozen BERT followed by an MLP), structural neighborhoods, and a temporal existence vector that encodes when the edge appears in the dynamic graph. The two embeddings are then aligned in a shared latent space using a similarity-based distillation loss, so that the temporal GNN learns to approximate the LLM’s semantic understanding without requiring the LLM at inference time.

4.2.2 LLM for Temporal Graph Construction. Another GNN-centered approach is to use LLMs to generate or enhance the temporal graph on which a GNN operates. *LLM-Enhanced Dynamic Graph Neural Network (LED-GNN)* [92] implements this for time-series forecasting: an LLM parses unstructured text (such as news) to infer latent relations between entities (e.g., stocks). The resulting edges then define a dynamic interaction graph for news relations, which is combined with a spatiotemporal stock graph (learned from prices/volumes) that has static edges but dynamic node features. Both graphs, defined over the same stock nodes, are fed into a dynamic GNN based on GATv2-LSTM. The node representations from the news and stock graphs are fused via cross-attention and passed to an MLP for volume prediction.

4.2.3 LLM as Semantic Feature Encoder. Furthermore, LLMs can be used to supply additional high-level semantic features for a temporal GNN. For example, *DLG-IDS* [48] dynamically constructs a device interaction graph (nodes are sensors/actuators/hosts; edges are learned or engineered dependencies that evolve over time) from system logs and control flows. Then they use an LLM to extract semantic cues (e.g., intent component descriptions, attack types, or expert rules) from raw traffic. These are injected as node and edge features. A lightweight *spatio-temporal GNN (STGNN)* — optimized through sparse attention and local window Transformers and thereby only having linear complexity — then processes the evolving graph. Inference therefore remains graph-based and efficient, but benefits from the LLM’s ability to provide context-aware feature representations.

4.3 Fusion Models

Fusion-based TGLHs lie between LLM-centered and GNN-centered models: neither component is just auxiliary. Unlike retrieval-centric or prompt-only approaches, fusion architectures integrate information within the computation at the level of embeddings, hidden states, or training objectives. Instead of treating one component as a fixed feature extractor

or teacher, both sides contribute signals that are combined at the representation, prediction, or loss level.

4.3.1 Token-level Fusion. Token-level fusion converts graph embeddings into token-like representations that are processed inside the LLM’s transformer layers. For example, the framework *GenTKGQA* proposed by Gao et al. [19], addresses the *Temporal Knowledge Graph Question Answering (TKGQA)* task, which requires reasoning over factual quadruples with explicit time intervals. *GenTKGQA* implements token-level fusion by mapping the graph’s structural and temporal information into virtual token embeddings. Specifically, it first encodes the retrieved subgraph using a TGNN, which captures both neighborhood connectivity and chronological dependencies through time-aware message passing. The resulting entity, relation, and timestamp embeddings are then pooled and combined with learnable time-boundary embeddings to form dense, time-enriched vector representations. These enriched embeddings are projected into the LLM’s textual space and processed alongside natural language tokens during instruction tuning. This enables the model to jointly reason over semantic and temporal-graph dependencies through a unified representation.

The same philosophy is also seen in *TGL-LLM* [9]. This framework aims to improve forecasting accuracy in the *Temporal Knowledge Graph Forecasting (TKGF)* task by enhancing the modeling of temporal evolution patterns and graph-language alignment. It employs a similar token-level fusion mechanism. Specifically, it utilizes a temporal GNN to encode the chronological sequence of historical graph snapshots into time-series entity embeddings. These sequential embeddings are then projected into a stream of virtual tokens via a trainable adapter and, together with a learned feature token, are prepended to the instruction text as a chronological description of the query subject. This allows the LLM’s transformer layers to process the temporal dynamics of the graph as an ordered token sequence.

4.3.2 Downstream Embedding Fusion. Downstream embedding fusion keeps LLM and GNN as separate encoders and merges their vector outputs only in a task-specific prediction module. *Dynamic Global-Recent Adaptive Semantic Processing (DyGRASP)* [82] also targets DyTAGs. It uses an LLM in two complementary roles: An implicit reasoning module (for each node, its historical interactions are ordered by time, chunked via a sliding window, and fed to an LLM so that hidden states yield *recent* semantic features for all interactions) and an explicit reasoning module (segments the same history into coarse time periods and prompts the LLM to generate textual summaries to capture long-range *global* semantic evolution). These recent and global semantic features are then processed by dedicated Transformer-based *Recent Semantic (RS)* and *Global Semantic (GS)* layers, while a temporal GNN provides dynamic structural embeddings in a separate graph-structure layer. Finally, DyGRASP merges the

RS, GS, and structural representations via readout functions and a Merge MLP into a single integrated node embedding used for destination-node retrieval and future link prediction.

(*Hybrid Knowledge-Augmented News Recommender (HKNR)* [47] instead fuses LLM- and GNN-derived embeddings within a downstream ranking network. In the recall stage, a frozen LLaMA-2-7B encodes article texts into semantic embeddings used for nearest-neighbor retrieval. A temporal user-item graph is then constructed from interaction logs, and a multi-layer GCN with temporal attention produces dynamic user embeddings. In the ranking stage, the LLM-based article embeddings, graph-based user embeddings, and structured knowledge features (topics, entities) are combined through gated feature fusion and cross-feature interaction layers before being scored by a residual MLP.

Another example is *LLM-GNN Aggregation (LGA)* [102], which consists of a *Long-Distance Interaction Capture Module* (an R-GCN that aggregates multi-hop structural and attribute information) and an *LLM Aggregation Module* (where a frozen LLM ranks a node’s incident links to produce an importance-weighted neighborhood embedding). The outputs of these two modules are concatenated to form the aggregated input sequence for each target node, which is then fed into a *gated recurrent unit (GRU)* to capture temporal dependencies and generate the predicted embedding for temporal evolution attribute graph prediction.

4.3.3 Score-level Fusion. Score-level (or prediction-level) fusion keeps the LLM and graph model as separate predictors and combines their outputs only at the last stage, e.g., by reweighting or interpolating their scores. An example is the already mentioned CoH [86], where the LLM performs chain-of-history reasoning and outputs a ranked list of candidate entities. These ranks are transformed into numeric scores via exponential decay, set to zero for unmentioned entities, and then linearly combined with a graph model’s scores to form the final prediction.

4.3.4 Training-level Fusion. LLMs and GNNs can also be combined through the training objective. A recent example is *GRAPHIA* [31], which uses dynamic social graphs as supervision signals for post-training LLM-based agents via *reinforcement learning (RL)*. A temporal GNN is trained on the graph to provide structural priors over interaction categories (i.e., edge types). Its logits and graph-derived statistics are converted into rewards for RL updates of the LLM policies. Consequently, the LLM gradually internalizes the temporal interaction patterns of the graph. At inference time, only the LLM agents are executed: conditioned on past interactions, they autoregressively generate future neighbors, edge types, and messages, thereby simulating the temporal evolution of the social graph without requiring the GNN anymore.

Table 1. Taxonomy of Temporal GNN-LLM Hybrids (TGLHs) benchmarks, categorized by their task category.

Task Category	Benchmarks / Datasets	Metrics
Temporal Question Answering	CronQuestions [71], MultiTQ [12], TimeQuestions [32]	Hits@1, Hits@10, Exact Match (EM), F1 Score
Temporal Graph Forecasting & Completion	ICEWS [7], GDELT [41], YAGO11k [16], WIKIDATA [40]	Mean Reciprocal Rank (MRR), Hits@1, Hits@3, Hits@10
Temporal Graph Generation	ProScript [53], Distantly Supervised TKG [105], IDGG/TDGG [31]	Graph Edit Distance (GED), Graph Isomorphism, Distributional Statistics (MMD)
General Temporal Reasoning	TimeBench [14]	Accuracy, F1 Score

5 Benchmarks & Evaluation for Temporal GNN-LLM Hybrids

There are various benchmarks and metrics used for evaluating TGLHs, specifically designed to assess temporal reasoning abilities. In this section, we categorize relevant benchmarks by their objective: temporal question answering, temporal forecasting, and temporal graph generation. We distinguish between *benchmarks* (standardized datasets with train/test splits) and *metrics* (mathematical functions used to score performance). For an overview, see Table 1.

5.1 Temporal Question Answering

The task of *Temporal Question Answering (TQA)* requires a model to answer natural language questions that involve explicit timestamps (e.g., “Who was the US President in 2012?”) or implicit temporal ordering (e.g., “Who held office before Obama?”) [71].

Datasets. Relevant datasets for temporal question answering include:

- *CronQuestions* [71]: A large-scale TKGQA dataset (410k QA pairs) constructed from Wikidata. It is essential for training models on complex reasoning, including comparisons and time-interval aggregations.
- *MultiTQ* [12]: Derived from the *Integrated Crisis Early Warning System (ICEWS)*. It is currently the largest TKGQA benchmark and is critical for testing specific timestamp granularity (e.g., year vs. month).
- *TimeQuestions* [32]: A human-curated dataset of approximately 16,000 examples featuring explicit, implicit, and ordinal temporal nuances, testing linguistic variety.

Metrics. Evaluation typically treats the answer as a retrieval or classification task over the set of entities. The primary metrics are *Hits@1* and *Hits@10*, which measure the percentage of questions where the correct answer appears

in the top- k predicted entities. for generative hybrids that output free text, *Exact Match (EM)* and *F1 Score* are used to compare the generated string against the gold-standard answer tokens.

5.2 Temporal Forecasting & Completion

Temporal forecasting and temporal completion involve predicting missing links or future states in a graph. Formally, given a history of facts up to time t , the model must predict the missing object ($s, r, ?, t + \delta$) or relation ($s, ?, o, t + \delta$) [36].

Datasets. Datasets for link prediction on KGs include:

- *ICEWS & GDELT*: These are event-centric datasets extracted from news corpora. ICEWS [7] focuses on political events with specific timestamps and is generally cleaner. GDELT [41] is significantly larger but noisier, often resulting in lower model performance due to abstract entity alignment issues.
- *YAGO11k & WIKIDATA*: Derived from standard knowledge bases, these datasets [16, 40] typically use time intervals (e.g., occursSince 2013) rather than instantaneous timestamps, providing richer duration information for testing interval-based reasoning.

Metrics. Since this can be formulated as a ranking task, performance can be evaluated using *Mean Reciprocal Rank (MRR)*, which calculates the average inverse rank of the true entity, and *Hits@k* (typically $k = 1, 3, 10$). In temporal settings, these metrics are often reported under “time-aware” filtering settings, where valid historical facts are not counted as false positives.

5.3 Temporal Graph Generation

Generative tasks evaluate the model’s ability to construct a graph topology from unstructured text or to generate plausible future graph snapshots. This corresponds to the *Temporal Graph Generation (TGG)* task.

Datasets. Datasets for evaluating TGG include:

- *ProScript* [53]: Originally designed for commonsense script learning, this dataset contains procedural event sequences (e.g., “baking a cake”) that require chronological ordering. It is widely adapted as a proxy benchmark for evaluating an LLM’s ability to generate valid temporal graph structures.
- *Distantly Supervised TKG Generation*: To address the scarcity of labeled data for generative tasks, Zhu et al. [105] introduced a dataset supervised distantly by LLMs, specifically designed to benchmark the generation of temporal knowledge graphs from raw text.
- *IDGG & TDGG*: Proposed by Ji et al. [31], these benchmarks evaluate Inductive and Transductive Dynamic Graph Generation, focusing on aligning emergent network properties and node-wise interactions respectively.

Metrics. Evaluating a generated graph requires comparing it to a ground truth structure:

- **Topology and Ordering:** *Graph Edit Distance (GED)* quantifies the minimum number of operations to transform the predicted graph into the ground truth [1]. While holistic, it is computationally expensive. Scalable alternatives include *Graph Isomorphism (ISO)* checks for small subgraphs and *Edge F1 Score*, which treats edge existence as a binary classification problem.
- **Distributional Quality:** For models generating entire graph snapshots, simple overlap metrics are often insufficient. Souid et al. [72] propose evaluating the *temporal evolution of graph statistics* (e.g., Degree, Clustering Coefficient, Centrality) using *Dynamic Time Warping (DTW)* to measure alignment between the generated and real distributions.

5.4 Other Datasets & Benchmarks

General Temporal Reasoning. While not graph-specific, *TimeBench* [14] is a comprehensive suite aggregating diverse datasets for tasks like temporal relation extraction and ordering. It serves as a holistic test of the foundational temporal reasoning abilities required by the LLM component of a hybrid. Evaluation primarily relies on **Accuracy** (for multiple choice) and **F1 Score** (for span extraction or relation labeling).

Continuous-Time Interaction Datasets. For evaluating dynamic GNN components specifically (e.g., TGN/TGAT), standard benchmarks focus on edge streams. The recent *Temporal Graph Benchmark (TGB)* [30] has standardized several datasets such as *Reddit*, *Wikipedia Edit History*, and *Bitcoin-OTC* [39]. These are crucial for hybrids that rely on identifying anomalous temporal patterns or community evolution.

5.5 Further Evaluation Considerations

Beyond standard graph or text metrics, TGLHs face unique evaluation challenges — specifically, the alignment between symbolic graph structures and neural textual representations. Future evaluation frameworks must address the following critical dimensions:

- **Temporal-Factual Consistency:** A key challenge is measuring whether the generated answer respects both semantic truth and temporal constraints. Metrics must distinguish between retrieving a fact that is semantically relevant but temporally expired versus one that is valid at the query time [99].
- **Structural vs. Semantic Trade-offs:** Hybrids often face an optimization trade-off where improving structural graph accuracy (e.g., minimizing Graph Edit Distance) may degrade the natural language fluency or reasoning flexibility of the LLM. Evaluation should report *Pareto frontiers* rather than single-metric scores.
- **Efficiency & Scalability:** Given the computational cost of GNNs and the latency of LLMs, metrics regarding inference time and memory consumption are critical. This includes evaluating the overhead of graph updates vs. the latency of token generation.
- **Privacy & Memorization:** As noted by Souid et al. [72], generative graph models risk memorizing training topologies. Evaluation should utilize privacy metrics, such as temporal extensions of the *Nearest Neighbor Distance Ratio (NNDR)*, to ensure models generate novel structures rather than leaking sensitive training interactions.

To evaluate these dimensions, recent work has moved beyond simple QA datasets toward complex reasoning benchmarks. For instance, Chen et al. [10] introduced a dataset specifically for time-sensitive question answering, requiring models to reason about when facts are valid. Similarly, benchmarks like *TempReason* [74] isolate temporal reasoning capabilities in LLMs, while *MultiHop-RAG* [75] benchmarks the system’s ability to retrieve and synthesize evidence across diverse multi-hop reasoning chains.

6 Performance & Costs of Temporal GNN-LLM Hybrids

In this section we shortly elaborate on the performance of recent TGLHs on some of the benchmarks mentioned in section 5, and furthermore highlight the limitations of the three hybrid paradigms (see section 4) regarding scalability and costs.

6.1 State-of-the-Art Performance

Recent benchmarks demonstrate that TGLH architectures significantly outperform unimodal baselines in tasks like temporal question answering. For example, the *TimeR⁴* framework demonstrated a 47.8% relative improvement in Hits@1

Table 2. Qualitative comparison of LLM-centered, GNN-centered, and fusion hybrids in terms of training cost, inference latency, and scalability constraints.

Paradigm	Training Cost	Inference Latency	Scalability	Primary Constraint
LLM-Centered	Low to High	High	Low	Context window limits (Quadratic cost)
GNN-Centered	Medium	Low	Medium	Graph size (Memory overhead)
Fusion	High	Medium to High	Low	Compound costs (Sequence length + Graph)

on standard datasets compared to static retrieval baselines [62]. Similarly, TempAgent achieved 70.2% Hits@1 on MultiTQ, a 41.3% improvement over baseline LLM agents lacking temporal tools [25].

Despite these successes, challenges remain in generative tasks. Results on ProScript reveal that while prompting techniques like Narrative-of-Thought improve LLM performance (boosting F1 by 16-71%), even powerful models like GPT-4 lag behind human baselines in complex temporal ordering tasks [100]. This suggests that while hybrids excel at retrieving explicit temporal facts, implicit temporal reasoning and graph construction remain active research frontiers.

6.2 Comparative Costs & Scalability

Even though TGLHs often outperform pure LLMs or pure GNNs on temporal tasks, they face limitations regarding training costs, inference latency, and scalability (see table 2).

6.2.1 Complexity of LLM-Centered Hybrids. In LLM-centered models, the large language model is the main inference engine and graphs serve as retrieved context or input to prompts. For example, EventRAG [95] uses an LLM to extract an event knowledge graph from text, and Sun et al. [73] introduce a “Time-CoT” prompt structure for guiding LLM reasoning with explicit temporal relations. While training costs vary (low for frozen RAG/prompting, high for fine-tuning), these approaches consistently incur high inference latency due to sequential token generation, and multiple LLM-calls. Another key bottleneck is the LLM’s limited context window (quadratic self-attention makes long sequences infeasible), so very long historical graphs must be compressed or truncated. In summary, LLM-centered systems are powerful but often have high inference latency and are constrained by context-length limits.

6.2.2 Complexity of GNN-Centered Hybrids. GNN-centered models use a dynamic graph neural network as the primary predictor, with any LLM support used offline (e.g., for supervision or graph construction). For instance, LKD4DyTAG [68] uses an LLM to generate textual edge representations that are distilled into a smaller temporal GNN, and LED-GNN [92] uses an LLM to infer latent edges which define a dynamic graph for a GNN-based forecaster. Once trained, these models rely only on the (much smaller) GNN at test time. Thus, training cost is medium (requiring an

LLM teacher or encoder), but inference latency is low (a single GNN forward pass). The main scalability bottleneck is graph size: very large or fast-evolving graphs demand more memory and may require sampling. In practice, certain GNN designs (e.g., linear or local-window spatio-temporal layers) can mitigate this. Overall, GNN-centered hybrids trade off semantic richness for faster, more scalable inference.

6.2.3 Complexity of Fusion Hybrids. Fusion architectures jointly integrate GNN and LLM components in their core computation. Training such models is typically high cost, since both the language and graph modules (and any fusion layers) must be trained or fine-tuned together. For example, token-level fusion methods like GenTKGQA [19] and TGL-LLM [9] encode subgraphs into virtual tokens that are injected into the LLM. Inference also has a higher latency than a pure GNN or a pure LLM, since each query requires running a graph encoder and then an LLM. Memory and scalability are challenged on both fronts: the LLM still has a bounded context window, and the GNN still must process its subgraph. In effect, fusion models inherit the bottlenecks of both paradigms. Requiring repeated graph updates and LLM inferences at fine time scales can quickly become prohibitive for large-scale or continuous-time graphs. Thus, fusion hybrids often have the heaviest computational footprint, albeit with the richest representational power.

7 Discussion

In this section, we place our survey in comparison to related work (see section 7.1), expand on core challenges of dynamic integration (see section 7.2), and provide an overview over possible future research directions (see section 7.3), before concluding this survey in section 7.4.

7.1 Related Work

There exist surveys that examine the use of temporal graphs and GNN-LLM hybrids in domain-specific settings [34, 51]. Beyond these targeted applications, several recent surveys provide broader overviews of GNN-LLM hybrids [65, 81], including perspectives framed under Graph Foundation Models (GFM) [49, 83]. Complementing this line of work, Arroyo et al. [4] discusses conceptual parallels between GNNs and LLMs, while Wu et al. [84] surveys the role of GNNs in NLP more generally. In parallel, many recent surveys focus on temporal and dynamic graphs [20, 33, 103].

However, the scopes of GNN-LLM hybrid surveys and those of temporal architectures remain largely orthogonal: surveys on dynamic GNNs primarily emphasize structural evolution, temporal dependencies, and evaluation metrics for evolving graphs, but typically do not consider the integration of LLMs or semantic reasoning. In contrast, GNN-LLM surveys tend to focus on semantic grounding and reasoning on graph-structured data, while largely assuming static graph representations and neglecting temporal modeling. Similarly, surveys on LLM-KG integration [45, 58] emphasize knowledge fusion and question answering, often treating graphs as static snapshots.

To the best of our knowledge, this survey is the first to systematically analyze the intersection of these lines of work, with a particular focus on temporal reasoning and on how the semantic capabilities of LLMs can be integrated with the evolving structure of dynamic graphs.

7.2 Core Challenges in Dynamic Integration

Developing TGLH systems introduces unique challenges not present in static hybrids:

Entity Identity and Evolution. In static graphs, an entity is a fixed node. In temporal settings, an entity’s attributes and role may shift drastically (e.g., a “Senator” becoming a “President”). TGLHs must maintain identity consistency while allowing semantic representation updates. Furthermore, handling complex lifecycle events like entity splits (e.g., corporate spin-offs) and merges remains an underexplored challenge in current temporal graph construction models, as suggested by recent event-centric dynamic graph retrieval work [73].

Temporal Conflict Resolution. When retrieving information, the model encounters contradictory facts that are both “true” but at different times. Souid et al. [72] highlight that without explicit dynamic modeling, generative models often fail to capture realistic temporal transitions across snapshots, leading to structurally invalid graphs. This aligns with the limitations of standard GraphRAG, which largely fails to capture temporal intent [73], necessitating time-aware embedding spaces as seen in TimeR⁴ [62].

Context vs. Recency Bias. A major challenge is balancing long-term dependencies with the tendency of dynamic graphs to be driven by immediate recency. Huang et al. [29] investigate the potential of LLMs as temporal graph learners by serializing temporal graphs into text sequences. Their results suggest that while LLMs can effectively utilize recent neighbors (exhibiting a “recency bias”), they struggle to integrate long-term historical structural patterns without explicit guidance, highlighting the tension between recency and global temporal context.

Representation Misalignment. Current TGLHs typically combine pre-trained LLMs and GNNs that were trained independently, resulting in embedding spaces with different geometric properties. LLM token embeddings capture distributional semantics, while GNN node embeddings encode graph topology. This mismatch complicates fusion strategies; simple concatenation often loses information, while complex projection layers require careful alignment strategies to synchronize time-awareness across both modalities [79].

Ethical and Privacy Considerations. Temporal graphs inherently encode sensitive information such as individual trajectories, event sequences, and evolving relationships. TGLHs must address privacy concerns, particularly the risk of models memorizing specific interaction sequences. Souid et al. [72] demonstrate that evaluating privacy risks and memorization in temporal graph generation is essential to prevent the leakage of sensitive historical patterns as these systems move toward deployment.

7.3 Future Research Directions

Based on the gaps identified in current architectures, we propose key directions for future research:

Towards Closed-Loop Temporal Systems. Currently, most TGLH architectures operate unidirectionally: either graphs provide structural grounding for LLMs, or LLMs provide semantic extraction for graph modules, but the resulting state is rarely fed back into the system. Building on iterative approaches like EventRAG’s self-correction [95] and TempAgent’s toolbox [25], future research should advance towards **closed-loop temporal systems**. Concretely, we envision a recurring *Graph Construction* $\rightarrow$ *GNN Refinement* $\rightarrow$ *LLM Retrieval* loop: new observations trigger graph edits, a temporal GNN propagates these updates through the historical neighborhood, and the LLM immediately re-queries the revised structure for time-conditioned generation. Such feedback-driven hybrids would approximate a self-sustaining cycle of world modeling, capable of continuously adapting to streaming evidence.

Explainable Temporal Causality. TGLHs are particularly well-suited for answering *why*-questions, not just *what* or *when*. However, most existing benchmarks and architectures emphasize predictive objectives (e.g., forecasting, completion) and rarely evaluate causal explanations. Future architectures could leverage the GNN to trace causal paths in the graph history and use the LLM to translate these structural paths into natural language explanations (Abductive Reasoning). This could address the lack of causality evaluation in current benchmarks [100].

Unified Temporal Pre-training. Most existing TGLHs couple independently pre-trained LLMs and GNNs, resulting in a representational mismatch between language-centric semantic spaces and graph-centric structural encodings. Joint

pre-training on aligned, time-stamped text–graph pairs offers a promising alternative: by exposing both components to shared temporal signals during training, hybrids could develop native temporal–structural representations rather than relying on post-hoc alignment. Such pre-training may substantially improve zero-shot temporal reasoning and reduce the need for complex fusion or projection layers at inference time.

Robust Alignment Strategies. To overcome the geometric mismatch between distribution-based LLM embeddings and topology-based GNN embeddings, future research should develop advanced alignment objectives beyond simple linear projections. For example, the TimeR framework fine-tunes a retriever using a contrastive time-aware learning objective, where timestamps are corrupted to generate hard negative samples that force the model to distinguish semantically similar but temporally distinct facts [62]. Similarly, LKD4DyTAG employs a similarity-based distillation loss to align a lightweight GNN’s representations with a teacher LLM’s semantic embeddings [68]. We suggest a hybrid approach: use contrastive objectives to align the “time” dimension (temporal validity) and distillation to align the “semantic” dimension (textual meaning) simultaneously, ensuring the GNN captures both evolving structure and linguistic nuance.

Standardized Evaluation Frameworks. Current evaluation lacks unified standards for jointly assessing structural, semantic, and temporal reasoning. As discussed in Section 5, most existing benchmarks either focus on fact retrieval or structural prediction, but rarely test causal temporal reasoning. Future work should develop evaluation suites that measure temporal consistency, robustness to timestamp noise, and the ability to distinguish correlation from causation.

Scalability. For real-time deployment or streaming settings, scalability remains a fundamental bottleneck for most TGLHs. Many architectures require repeated GNN updates and LLM inference at fine-grained temporal resolutions, which is prohibitive for large-scale or continuous-time graphs. Promising directions include event-triggered graph updates instead of full recomputation, or lightweight distillation of LLM reasoning into graph modules, and asynchronous execution where LLM reasoning operates at coarser temporal intervals than graph updates.

7.4 Conclusion

Temporal GNN–LLM hybrids mark a substantive shift in neuro-symbolic AI, moving beyond static knowledge representations toward dynamic world models. By systematically categorizing these architectures, we expose the emerging design paradigms for developing models that do not merely accumulate historical data, but reason causally over evolving structures. The field is still an early stage — constrained by unresolved issues in scalability, entity management, and

the trade-off between recency and long-term context. However, the potential for TGLHs to enable self-updating, time-aware, and causally explainable artificial intelligence makes them one of the most promising frontiers in graph learning. Progress will require not only novel architectures but also standardized evaluation frameworks that jointly assess structural, semantic, and temporal reasoning capabilities.

Limitations

Scope. Our survey focuses on architectural integration and core methodologies; domain-specific applications (e.g., finance, healthcare) are discussed illustratively but not exhaustively. Furthermore, this work does not aim to be a systematic literature review and therefore cannot claim exhaustive coverage of all relevant literature in general.

Consideration of Costs & Scalability. We did neither evaluate the computational costs nor the runtime complexity of the different architectures, which would be necessary to consider for real-life applications.

References

- [1] Zeina Abu-Aisheh, Romain Raveaux, Jean-Yves Ramel, and Patrick Martineau. 2015. An Exact Graph Edit Distance Algorithm for Solving Pattern Recognition Problems. In *Proceedings of the International Conference on Pattern Recognition Applications and Methods - Volume 1* (Lisbon, Portugal) (ICPRAM 2015). SCITEPRESS - Science and Technology Publications, Lda, Setubal, PRT, 271–278. <https://doi.org/10.5220/0005209202710278>
- [2] James F. Allen. 1983. Maintaining knowledge about temporal intervals. *Commun. ACM* 26, 11 (Nov. 1983), 832–843. <https://doi.org/10.1145/182.358434>
- [3] Uri Alon and Eran Yahav. 2021. On the Bottleneck of Graph Neural Networks and its Practical Implications. In *International Conference on Learning Representations*. <https://openreview.net/forum?id=i80OPhOCVH2>
- [4] Álvaro Arroyo, Federico Barbero, Hugh Blayney, Michael Bronstein, Xiaowen Dong, Pietro Liò, Razvan Pascanu, and Pierre Vanderghelynst. 2025. Bridging Graph Neural Networks and Large Language Models: A Survey and Unified Perspective. (nov 2025). <https://infoscience.epfl.ch/handle/20.500.14299/255480>
- [5] Dmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. 2015. Neural Machine Translation by Jointly Learning to Align and Translate. In *3rd International Conference on Learning Representations, ICLR 2015, San Diego, CA, USA, May 7-9, 2015, Conference Track Proceedings*, Yoshua Bengio and Yann LeCun (Eds.). <http://arxiv.org/abs/1409.0473>
- [6] Eden Biran, Daniela Gottesman, Sohee Yang, Mor Geva, and Amir Globerson. 2024. Hopping Too Late: Exploring the Limitations of Large Language Models on Multi-Hop Queries. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen (Eds.). Association for Computational Linguistics, Miami, Florida, USA, 14113–14130. <https://doi.org/10.18653/v1/2024.emnlp-main.781>
- [7] Elizabeth Boschee, Jennifer Lautenschlager, Sean O’Brien, Steve Shellman, James Starz, and Michael Ward. 2015. ICEWS Coded Event Data. <https://doi.org/10.7910/DVN/28075>
- [8] Borui Cai, Yong Xiang, Longxiang Gao, He Zhang, Yunfeng Li, and Jianxin Li. 2023. Temporal knowledge graph completion: a survey (IJCAI ’23). Article 734, 9 pages. <https://doi.org/10.24963/ijcai.2023/734>

- [9] He Chang, Jie Wu, Zhulin Tao, Yunshan Ma, Xianglin Huang, and Tat-Seng Chua. 2025. Integrate Temporal Graph Learning into LLM-based Temporal Knowledge Graph Model. *arXiv:2501.11911* [cs.IR] <https://arxiv.org/abs/2501.11911>
- [10] Wenhui Chen, Xinyi Wang, and William Yang Wang. 2021. A Dataset for Answering Time-Sensitive Questions. In *Thirty-fifth Conference on Neural Information Processing Systems Datasets and Benchmarks Track (Round 2)*. <https://openreview.net/forum?id=9-LSfSU74n->
- [11] Yakun Chen, Xianzhi Wang, and Guandong Xu. 2023. GATGPT: A Pre-trained Large Language Model with Graph Attention Network for Spatiotemporal Imputation. *arXiv:2311.14332* [cs.LG] <https://arxiv.org/abs/2311.14332>
- [12] Ziyang Chen, Jinzhi Liao, and Xiang Zhao. 2023. Multi-granularity Temporal Question Answering over Knowledge Graphs. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Anna Rogers, Jordan Boyd-Graber, and Naoaki Okazaki (Eds.). Association for Computational Linguistics, Toronto, Canada, 11378–11392. <https://doi.org/10.18653/v1/2023.acl-long.637>
- [13] Zhikai Chen, Haitao Mao, Hang Li, Wei Jin, Hongzhi Wen, Xiaochi Wei, Shuaiqiang Wang, Dawei Yin, Wenqi Fan, Hui Liu, and Jiliang Tang. 2024. Exploring the Potential of Large Language Models (LLMs) in Learning on Graphs. *SIGKDD Explor. Newsl.* 25, 2 (March 2024), 42–61. <https://doi.org/10.1145/3655103.3655110>
- [14] Zheng Chu, Jingchang Chen, Qianglong Chen, Weijiang Yu, Haotian Wang, Ming Liu, and Bing Qin. 2024. TimeBench: A Comprehensive Evaluation of Temporal Reasoning Abilities in Large Language Models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Lun-Wei Ku, Andre Martins, and Vivek Srikumar (Eds.). Association for Computational Linguistics, Bangkok, Thailand, 1204–1228. <https://doi.org/10.18653/v1/2024.acl-long.66>
- [15] Gabriele Corso, Hannes Stark, Stefanie Jegelka, Tommi Jaakkola, and Regina Barzilay. 2024. Graph neural networks. *Nature Reviews Methods Primers* 4, 1 (2024), 17.
- [16] Shib Sankar Dasgupta, Swayambhu Nath Ray, and Partha Talukdar. 2018. HyTE: Hyperplane-based Temporally aware Knowledge Graph Embedding. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*. 2001–2011.
- [17] Bhuwan Dhingra, Jeremy R. Cole, Julian Martin Eisenschlos, Daniel Gillick, Jacob Eisenstein, and William W. Cohen. 2022. Time-Aware Language Models as Temporal Knowledge Bases. *Transactions of the Association for Computational Linguistics* 10 (2022), 257–273. https://doi.org/10.1162/tacl_a_00459
- [18] M. F. C. dos Santos, Zheng Zhao, B. P. C. dos Santos, and Yu Wang. 2024. Let Your Graph Do the Talking: Encoding Graph Structure in Language Models. In *Findings of the Association for Computational Linguistics: EMNLP 2024*.
- [19] Yifu Gao, Linbo Qiao, Zhigang Kan, Zhihua Wen, Yongquan He, and Dongsheng Li. 2024. Two-stage Generative Question Answering on Temporal Knowledge Graph Using Large Language Models. In *Findings of the Association for Computational Linguistics: ACL 2024*, Lun-Wei Ku, Andre Martins, and Vivek Srikumar (Eds.). Association for Computational Linguistics, Bangkok, Thailand, 6719–6734. <https://doi.org/10.18653/v1/2024.findings-acl.401>
- [20] Shubham Gupta and Srikanth Bedathur. 2022. A survey on temporal graph representation learning and generative modeling. *arXiv preprint arXiv:2208.12126* (2022).
- [21] William L. Hamilton, Rex Ying, and Jure Leskovec. 2017. Inductive Representation Learning on Large Graphs. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 30. 1024–1034.
- [22] Xiaoxin He, Xavier Bresson, Thomas Laurent, Adam Perold, Yann LeCun, and Bryan Hooi. 2024. Harnessing Explanations: LLM-to-LLM Interpreter for Enhanced Text-Attributed Graph Representation Learning. In *The Twelfth International Conference on Learning Representations*. <https://openreview.net/forum?id=RXFVcynVe1>
- [23] Aidan Hogan, Eva Blomqvist, Michael Cochez, Claudia D’amato, Gerard De Melo, Claudio Gutierrez, Sabrina Kirrane, José Emilio Labra Gayo, Roberto Navigli, Sebastian Neumaier, Axel-Cyrille Ngonga Ngomo, Axel Polleres, Sabbir M. Rashid, Anisa Rula, Lukas Schmelzeisen, Juan Sequeda, Steffen Staab, and Antoine Zimmermann. 2021. Knowledge Graphs. *Comput. Surveys* 54, 4 (jul 2021), 1–37. <https://doi.org/10.1145/3447772>
- [24] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, Weizhu Chen, et al. 2022. Lora: Low-rank adaptation of large language models. *ICLR* 1, 2 (2022), 3.
- [25] Qianyi Hu, Xinhui Tu, Cong Guo, and Shunping Zhang. 2025. Time-aware ReAct Agent for Temporal Knowledge Graph Question Answering. In *Findings of the Association for Computational Linguistics: NAACL 2025*, Luis Chiruzzo, Alan Ritter, and Lu Wang (Eds.). Association for Computational Linguistics, Albuquerque, New Mexico, 6013–6024. <https://doi.org/10.18653/v1/2025.findings-naacl.334>
- [26] Shengxiang Hu, Guobing Zou, Song Yang, Shiyi Lin, Yanglan Gan, Bofeng Zhang, and Yixin Chen. 2025. Large Language Model Meets Graph Neural Network in Knowledge Distillation. *Proceedings of the AAAI Conference on Artificial Intelligence* 39, 16 (Apr. 2025), 17295–17304. <https://doi.org/10.1609/aaai.v39i16.33901>
- [27] Xiping Hu, Terry HS Chu, Victor CM Leung, Edith C-H Ngai, Philippe Kruchten, and Henry CB Chan. 2014. A survey on mobile social networks: Applications, platforms, system architectures, and future research directions. *IEEE Communications Surveys & Tutorials* 17, 3 (2014), 1557–1581.
- [28] Lei Huang, Weijiang Yu, Weitao Ma, Weihong Zhong, Zhangyin Feng, Haotian Wang, Qianglong Chen, Weihua Peng, Xiaocheng Feng, Bing Qin, and Ting Liu. 2025. A Survey on Hallucination in Large Language Models: Principles, Taxonomy, Challenges, and Open Questions. *ACM Trans. Inf. Syst.* 43, 2, Article 42 (Jan. 2025), 55 pages. <https://doi.org/10.1145/3703155>
- [29] Shenyang Huang, Emma Kondrup, Ali Parviz, Zachary Yang, Zifeng Ding, Michael M. Bronstein, Reihaneh Rabbany, and Guillaume Rabusseau. 2025. Are Large Language Models Good Temporal Graph Learners?. In *New Perspectives in Graph Machine Learning*. <https://openreview.net/forum?id=4ZithWQFQN>
- [30] Shenyang Huang, Farimah Poursafaei, Jacob Danovitch, Matthias Fey, Weihua Hu, Emanuele Rossi, Jure Leskovec, Michael Bronstein, Guillaume Rabusseau, and Reihaneh Rabbany. 2023. Temporal graph benchmark for machine learning on temporal graphs (*NIPS ’23*). Curran Associates Inc., Red Hook, NY, USA, Article 99, 18 pages.
- [31] Jiarui Ji, Zehua Zhang, Zhewei Wei, Bin Tong, Guan Wang, and Bo Zheng. 2025. GRAPHIA: Harnessing Social Graph Data to Enhance LLM-Based Social Simulation. *arXiv:2510.24251* [cs.SI] <https://arxiv.org/abs/2510.24251>
- [32] Zhen Jia, Soumajit Pramanik, Rishiraj Saha Roy, and Gerhard Weikum. 2021. Complex Temporal Question Answering on Knowledge Graphs. In *Proceedings of the 30th ACM International Conference on Information & Knowledge Management (Virtual Event, Queensland, Australia) (CIKM ’21)*. Association for Computing Machinery, New York, NY, USA, 792–802. <https://doi.org/10.1145/3459637.3482416>
- [33] Pengfei Jiao, Hongjiang Chen, Xuan Guo, Zhidong Zhao, Dongxiao He, and Di Jin. 2025. A survey on temporal interaction graph representation learning: progress, challenges, and opportunities. In *Proceedings of the Thirty-Fourth International Joint Conference on Artificial Intelligence (Montreal, Canada) (IJCAI ’25)*. Article 1166, 9 pages. <https://doi.org/10.24963/ijcai.2025/1166>
- [34] Guangyin Jin, Yuxuan Liang, Yuchen Fang, Zezhi Shao, Jincan Huang, Junbo Zhang, and Yu Zheng. 2024. Spatio-Temporal Graph Neural Networks for Predictive Learning in Urban Computing: A Survey. *IEEE Trans. on Knowl. and Data Eng.* 36, 10 (Oct. 2024), 5388–5408.

- <https://doi.org/10.1109/TKDE.2023.3333824>
- [35] Seyed Mehran Kazemi, Rishab Goel, Kshitij Jain, Ivan Kobzyev, Akshay Sethi, Peter Forsyth, and Pascal Poupert. 2020. Representation learning for dynamic graphs: a survey. *J. Mach. Learn. Res.* 21, 1, Article 70 (Jan. 2020), 73 pages.
- [36] Seyed Mehran Kazemi, Rishab Goel, Kshitij Jain, Ivan Kobzyev, Akshay Sethi, Peter Forsyth, and Pascal Poupert. 2020. Representation learning for dynamic graphs: A survey. *Journal of Machine Learning Research* 21, 70 (2020), 1–73.
- [37] Thomas N. Kipf and Max Welling. 2017. Semi-Supervised Classification with Graph Convolutional Networks. In *International Conference on Learning Representations (ICLR)*. <https://openreview.net/forum?id=SJU4ayYgl>
- [38] Elahe Kooshafar. 2025. Representation Integrity in Temporal Graph Learning Methods. *arXiv preprint arXiv:2511.20873* (2025).
- [39] Srikanth Kumar, Xikun Zhang, and Jure Leskovec. 2019. Predicting Dynamic Embedding Trajectory in Temporal Interaction Networks (KDD '19). Association for Computing Machinery, New York, NY, USA, 1269–1278. <https://doi.org/10.1145/3292500.3330895>
- [40] Julien Leblay and Melisachew Wudage Chekol. 2018. Deriving Validity Time in Knowledge Graph. In *Companion Proceedings of the The Web Conference 2018*. 1771–1776. <https://doi.org/10.1145/3184558.3191639>
- [41] Kalev Leetaru and Philip A. Schrodt. 2013. GDELT: Global Data on Events, Location, and Tone. In *ISA Annual Convention*, Vol. 2. 1–49. <http://data.gdeltproject.org/documentation/ISA.2013.GDELT.pdf>
- [42] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *Advances in Neural Information Processing Systems*, H. Larochelle, M. Ranzato, R. Hadsell, M.F. Balcan, and H. Lin (Eds.), Vol. 33. Curran Associates, Inc., 9459–9474. https://proceedings.neurips.cc/paper_files/paper/2020/file/6b493230205f780e1bc26945df7481e5-Paper.pdf
- [43] Dong Li, Yichen Niu, Ying Ai, Xiang Zou, Bqing Qi, and Jianxing Liu. 2025. T-GRAG: A Dynamic GraphRAG Framework for Resolving Temporal Conflicts and Redundancy in Knowledge Retrieval. In *Proceedings of the 33rd ACM International Conference on Multimedia (Dublin, Ireland) (MM '25)*. Association for Computing Machinery, New York, NY, USA, 11880–11889. <https://doi.org/10.1145/3746027.3755628>
- [44] Mengran Li, Pengyu Zhang, Wenbin Xing, Yijia Zheng, Klim Zaporozhets, Junzhou Chen, Ronghui Zhang, Yong Zhang, Siyuan Gong, Jia Hu, Xiaolei Ma, Zhiyuan Liu, Paul Groth, and Marcel Worring. 2026. A Survey of Large Language Models for Data Challenges in Graphs. *Expert Systems with Applications* 298 (1 March 2026). <https://doi.org/10.1016/j.eswa.2025.129643> Publisher Copyright: © 2025 Elsevier Ltd.
- [45] Yuhang Li, Zhixun Li, Peisong Wang, Jia Li, Xiangguo Sun, Hong Cheng, and Jeffrey Xu Yu. 2024. A survey of graph meets large language model: progress and future directions. In *Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence (Jeju, Korea) (IJCAI '24)*. Article 898, 9 pages. <https://doi.org/10.24963/ijcai.2024/898>
- [46] Haochen Liu, Song Wang, and Jundong Li. 2025. Knowledge Graph Retrieval-Augmented Generation via GNN-Guided Prompting. In *Second Conference on Language Modeling*. <https://openreview.net/forum?id=R1NWMExESj>
- [47] Junchen Liu. 2025. Knowledge-Augmented News Recommendation via LLM Recall, Temporal GNN Encoding, and Multi-Task Ranking. 141–144. <https://doi.org/10.1109/ICBASE66587.2025.11181275>
- [48] Junyi Liu, Jiarong Wang, Tian Yan, Fazhi Qi, and Gang Chen. 2025. DLG-IDS: Dynamic Graph and LLM-Semantic Enhanced Spatiotemporal GNN for Lightweight Intrusion Detection in Industrial Control Systems. *Electronics* 14, 19 (2025). <https://doi.org/10.3390/electronics14193952>
- [49] Jiawei Liu, Cheng Yang, Zhiyuan Lu, Junze Chen, Yibo Li, Mengmei Zhang, Ting Bai, Yuan Fang, Lichao Sun, Philip S. Yu, and Chuan Shi. 2023. Towards Graph Foundation Models: A Survey and Beyond. *arXiv e-prints*, Article arXiv:2310.11829 (Oct. 2023), arXiv:2310.11829 pages. <https://doi.org/10.48550/arXiv.2310.11829> [cs.LG]
- [50] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. 2024. Lost in the Middle: How Language Models Use Long Contexts. *Transactions of the Association for Computational Linguistics* 12 (2024), 157–173. https://doi.org/10.1162/tac1_a_00638
- [51] Yang Lu, Shengli Zhang, Chang Liu, Ruichen Zhang, Bo Ai, Dusit Niyato, Wei Ni, Xianbin Wang, and Abbas Jamalipour. 2025. Agentic Graph Neural Networks for Wireless Communications and Networking Towards Edge General Intelligence: A Survey. arXiv:2508.08620 [eess.SP] <https://arxiv.org/abs/2508.08620>
- [52] Ruilin Luo, Tianle Gu, Haoling Li, Junzhe Li, Zicheng Lin, Jiayi Li, and Yujiu Yang. 2024. Chain of History: Learning and Forecasting with LLMs for Temporal Knowledge Graph Completion. arXiv:2401.06072 [cs.AI] <https://arxiv.org/abs/2401.06072>
- [53] Aman Madaan and Yiming Yang. 2021. Neural Language Modeling for Contextualized Temporal Graph Generation. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, Kristina Toutanova, Anna Rumshisky, Luke Zettlemoyer, Dilek Hakkani-Tur, Iz Beltagy, Steven Bethard, Ryan Cotterell, Tanmoy Chakraborty, and Yichao Zhou (Eds.). Association for Computational Linguistics, Online, 864–881. <https://doi.org/10.18653/v1/2021.naacl-main.67>
- [54] Othon Michail. 2015. *An Introduction to Temporal Graphs: An Algorithmic Perspective*. Springer International Publishing, Cham, 308–343. https://doi.org/10.1007/978-3-319-24024-4_18
- [55] Anthony N Michel. 2002. Recent trends in the stability analysis of hybrid dynamical systems. *IEEE Transactions on Circuits and Systems I: Fundamental Theory and Applications* 46, 1 (2002), 120–134.
- [56] Kenta Oono and Taiji Suzuki. 2020. Graph Neural Networks Exponentially Lose Expressive Power for Node Classification. In *International Conference on Learning Representations*. <https://openreview.net/forum?id=S1ldO2EFPr>
- [57] Bo Pan, Zheng Zhang, Yifei Zhang, Yuntong Hu, and Liang Zhao. 2024. Distilling Large Language Models for Text-Attributed Graph Learning. arXiv:2402.12022 [cs.CL] <https://arxiv.org/abs/2402.12022>
- [58] Zihan Pan, Zhiyu Li, Yifan Dai, Jun Ding, Rong Li, Dong Yu, Xin Xia, and Shirui Pan. 2024. A Survey on Augmenting Knowledge Graphs with Large Language Models. *arXiv preprint arXiv:2402.13848* (2024).
- [59] Aldo Pareja, Giacomo Domeniconi, Jie Chen, Tengfei Ma, Toyotaro Suzumura, Hiroki Kanezashi, Tim Kaler, Tao B. Schardl, and Charles E. Leiserson. 2020. EvolveGCN: Evolving Graph Convolutional Networks for Dynamic Graphs. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 34. 5363–5370. <https://doi.org/10.1609/aaai.v34i04.5984>
- [60] Boci Peng, Yun Zhu, Yongchao Liu, Xiaohe Bo, Haizhou Shi, Chuntao Hong, Yan Zhang, and Siliang Tang. 2025. Graph Retrieval-Augmented Generation: A Survey. *ACM Trans. Inf. Syst.* (Nov. 2025). <https://doi.org/10.1145/3777378> Just Accepted.
- [61] George Pu, Anirudh Jain, Jihan Yin, and Russell Kaplan. 2023. Empirical Analysis of the Strengths and Weaknesses of PEFT Techniques for LLMs. In *ICLR 2023 Workshop on Mathematical and Empirical Understanding of Foundation Models*.

- [62] Xinying Qian, Ying Zhang, Yu Zhao, Baohang Zhou, Xuhui Sui, Li Zhang, and Kehui Song. 2024. TimeR4: Time-aware retrieval-augmented large language models for temporal knowledge graph question answering. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. 6942–6952.
- [63] Ali Rahimi et al. 2024. Transformers are Graph Neural Networks. *arXiv preprint arXiv:2406.22084* (2024).
- [64] Mohaimenul Azam Khan Raiaan, Md Saddam Hossain Mukta, Kaniz Fatema, Nur Mohammad Fahad, Sadman Sakib, Most Marufatul Janan Mim, Jubaer Ahmad, Mohammed Eunus Ali, and Sami Azam. 2024. A review on large language models: Architectures, applications, taxonomies, open issues and challenges. *IEEE access* 12 (2024), 26839–26874.
- [65] Xubin Ren, Jiabin Tang, Dawei Yin, Nitesh Chawla, and Chao Huang. 2024. A Survey of Large Language Models for Graphs. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining* (Barcelona, Spain) (KDD '24). Association for Computing Machinery, New York, NY, USA, 6616–6626. <https://doi.org/10.1145/3637528.3671460>
- [66] Guy D. Rosin and Kira Radinsky. 2022. Temporal Attention for Language Models. In *Findings of the Association for Computational Linguistics: NAACL 2022*, Marine Carpuat, Marie-Catherine de Marneffe, and Ivan Vladimir Meza Ruiz (Eds.). Association for Computational Linguistics, Seattle, United States, 1498–1508. <https://doi.org/10.18653/v1/2022.findings-naacl.112>
- [67] Emanuele Rossi, Ben Chamberlain, Fabrizio Frasca, Davide Eyraud, Federico Monti, and Michael Bronstein. 2020. Temporal Graph Networks for Deep Learning on Dynamic Graphs. *arXiv:2006.10637* [cs.LG] <https://arxiv.org/abs/2006.10637>
- [68] Amit Roy, Ning Yan, and Masood Mortazavi. 2025. LLM-driven Knowledge Distillation for Dynamic Text-Attributed Graphs. *arXiv:2502.10914* [cs.LG] <https://arxiv.org/abs/2502.10914>
- [69] Pranab Sahoo, Prabhath Meharia, Akash Ghosh, Sriparna Saha, Vinija Jain, and Aman Chadha. 2024. A Comprehensive Survey of Hallucination in Large Language, Image, Video and Audio Foundation Models. In *Findings of the Association for Computational Linguistics: EMNLP 2024*, Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen (Eds.). Association for Computational Linguistics, Miami, Florida, USA, 11709–11724. <https://doi.org/10.18653/v1/2024.findings-emnlp.685>
- [70] Mansi Sakarvadia. 2024. Towards interpreting language models: A case study in multi-hop reasoning. *arXiv preprint arXiv:2411.05037* (2024).
- [71] Apoorv Saxena, Soumen Chakrabarti, and Partha Talukdar. 2021. Question Answering Over Temporal Knowledge Graphs. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, Chengqing Zong, Fei Xia, Wenjie Li, and Roberto Navigli (Eds.). Association for Computational Linguistics, Online, 6663–6676. <https://doi.org/10.18653/v1/2021.acl-long.520>
- [72] Housseem Eddine Souid, Lucas Ody, Youssef Achenchabe, Valentin Lemaire, Gianmarco Aversano, and Sabri Skhiri. 2024. Temporal Graph Generative Models: An empirical study. In *Proceedings of the 4th Workshop on Machine Learning and Systems* (Athens, Greece) (*EuroMLSys '24*). Association for Computing Machinery, New York, NY, USA, 18–27. <https://doi.org/10.1145/3642970.3655829>
- [73] Qingyun Sun, Jiaqi Yuan, Shan He, Xiao Guan, Haonan Yuan, Xingcheng Fu, Jianxin Li, and Philip S. Yu. 2025. DyG-RAG: Dynamic Graph Retrieval-Augmented Generation with Event-Centric Reasoning. *arXiv:2507.13396* [cs.IR] <https://arxiv.org/abs/2507.13396>
- [74] Qingyu Tan, Hwee Tou Ng, and Lidong Bing. 2023. Towards Benchmarking and Improving the Temporal Reasoning Capability of Large Language Models. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Anna Rogers, Jordan Boyd-Graber, and Naoaki Okazaki (Eds.). Association for Computational Linguistics, Toronto, Canada, 14820–14835. <https://doi.org/10.18653/v1/2023.acl-long.828>
- [75] Yixuan Tang and Yi Yang. 2024. MultiHop-RAG: Benchmarking Retrieval-Augmented Generation for Multi-Hop Queries. *arXiv:2401.15391* [cs.CL] <https://arxiv.org/abs/2401.15391>
- [76] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is All you Need. In *Advances in Neural Information Processing Systems*, I. Guyon, U. Von Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett (Eds.), Vol. 30. Curran Associates, Inc. https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf
- [77] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. 2018. Graph Attention Networks. In *International Conference on Learning Representations (ICLR)*. <https://openreview.net/forum?id=rJXMpikCZ>
- [78] Cunxiang Wang, Xiaozhe Liu, Yuanhao Yue, Qipeng Guo, Xiangkun Hu, Xiangru Tang, Tianhang Zhang, Cheng Jiayang, Yunzhi Yao, Xuming Hu, Zehan Qi, Wenyang Gao, Yidong Wang, Linyi Yang, Jindong Wang, Xing Xie, Zheng Zhang, and Yue Zhang. 2025. Survey on Factuality in Large Language Models. *ACM Comput. Surv.* 58, 1, Article 13 (Sept. 2025), 37 pages. <https://doi.org/10.1145/3742420>
- [79] Duo Wang, Yuan Zuo, Fengzhi Li, and Junjie Wu. 2024. LLMs as zero-shot graph learners: Alignment of gnn representations with llm token embeddings. *Advances in Neural Information Processing Systems* 37 (2024), 5950–5973.
- [80] Honggang Wang, Ye Li, Wenzhi Zhao, Haozhe Zhu, Jin Zhang, and Xuening Wu. 2025. GSF-LLM: Graph-Enhanced Spatio-Temporal Fusion-Based Large Language Model for Traffic Prediction. *Sensors* 25, 21 (2025). <https://doi.org/10.3390/s25216698>
- [81] Shijie Wang, Jiani Huang, Zhikai Chen, Yu Song, Wenzhuo Tang, Haitao Mao, Wenqi Fan, Hui Liu, Xiaorui Liu, Dawei Yin, and Qing Li. 2025. Graph Machine Learning in the Era of Large Language Models (LLMs). *ACM Trans. Intell. Syst. Technol.* 16, 5, Article 104 (Aug. 2025), 40 pages. <https://doi.org/10.1145/3732786>
- [82] Yunan Wang, Jianxin Li, and Ziwei Zhang. 2025. Global-Recent Semantic Reasoning on Dynamic Text-Attributed Graphs with Large Language Models. *arXiv:2509.18742* [cs.CL] <https://arxiv.org/abs/2509.18742>
- [83] Zehong Wang, Chuxu Zhang, Jundong Li, Nitesh Chawla, and Yanfang Ye. 2025. Graph Foundation Models: Challenges, Methods, and Open Questions. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.2* (Toronto ON, Canada) (KDD '25). Association for Computing Machinery, New York, NY, USA, 6184–6194. <https://doi.org/10.1145/3711896.3736568>
- [84] Lingfei Wu, Yu Chen, Kai Shen, Xiaojie Guo, Hanning Gao, Shucheng Li, Jian Pei, and Bo Long. 2023. Graph Neural Networks for Natural Language Processing: A Survey. *Foundations and Trends® in Machine Learning* 16, 2 (2023), 119–328. <https://doi.org/10.1561/22000000096>
- [85] Yuxia Wu, Lizi Liao, and Yuan Fang. 2025. Retrieval Augmented Generation for Dynamic Graph Modeling. In *Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval* (Padua, Italy) (SIGIR '25). Association for Computing Machinery, New York, NY, USA, 1434–1443. <https://doi.org/10.1145/3726302.3729958>
- [86] Yuwei Xia, Ding Wang, Qiang Liu, Liang Wang, Shu Wu, and Xiao-Yu Zhang. 2024. Chain-of-History Reasoning for Temporal Knowledge Graph Forecasting. In *Findings of the Association for Computational Linguistics: ACL 2024*, Lun-Wei Ku, Andre Martins, and Vivek Sriku-mar (Eds.). Association for Computational Linguistics, Bangkok, Thailand, 16144–16159. <https://doi.org/10.18653/v1/2024.findings-acl.955>
- [87] Siheng Xiong, Ali Payani, Ramana Kompella, and Faramarz Fekri. 2024. Large Language Models Can Learn Temporal Reasoning.

- In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Lun-Wei Ku, Andre Martins, and Vivek Srikumar (Eds.). Association for Computational Linguistics, Bangkok, Thailand, 10452–10470. <https://doi.org/10.18653/v1/2024.acl-long.563>
- [88] Da Xu, Chuanwei Ruan, Evren Körpeoglu, Sushant Kumar, and Kannan Achan. 2020. Inductive Representation Learning on Temporal Graphs. In *International Conference on Learning Representations (ICLR)*. <https://openreview.net/forum?id=rJeW1yHYwH>
- [89] Junjie Xu, Zongyu Wu, Minhua Lin, Xiang Zhang, and Suhang Wang. 2024. LLM and GNN are Complementary: Distilling LLM for Multimodal Graph Learning. arXiv:2406.01032 [cs.LG] <https://arxiv.org/abs/2406.01032>
- [90] Keyulu Xu, Weihua Hu, Jure Leskovec, and Stefanie Jegelka. 2019. How Powerful are Graph Neural Networks?. In *International Conference on Learning Representations*. <https://openreview.net/forum?id=ryGs6iA5Kmn>
- [91] Peng Xu, Wei Ping, Xianchao Wu, Lawrence McAfee, Chen Zhu, Zihan Liu, Sandeep Subramanian, Evelina Bakhturina, Mohammad Shoeybi, and Bryan Catanzaro. 2024. Retrieval meets Long Context Large Language Models. In *The Twelfth International Conference on Learning Representations*. <https://openreview.net/forum?id=xw5nxFWMlo>
- [92] Zhiyu Xu, Yi Liu, Yuchi Wang, Ruihan Bao, Keiko Harimoto, and Xu Sun. 2025. Modeling Interactions Between Stocks Using LLM-Enhanced Graphs for Volume Prediction. In *Proceedings of the Joint Workshop of the 9th Financial Technology and Natural Language Processing (FinNLP), the 6th Financial Narrative Processing (FNP), and the 1st Workshop on Large Language Models for Finance and Legal (LLMFinLegal)*, Chung-Chi Chen, Antonio Moreno-Sandoval, Jimin Huang, Qianqian Xie, Sophia Ananiadou, and Hsin-Hsi Chen (Eds.). Association for Computational Linguistics, Abu Dhabi, UAE, 153–163. <https://aclanthology.org/2025.finnlp-1.14/>
- [93] Haotong Yang, Xiyuan Wang, Qian Tao, Shuxian Hu, Zhouchen Lin, and Muhun Zhang. 2024. Gf-fusion: Rethinking the combination of graph neural network and large language model. *arXiv preprint arXiv:2412.06849* (2024).
- [94] Sohee Yang, Elena Gribovskaya, Nora Kassner, Mor Geva, and Sebastian Riedel. 2024. Do Large Language Models Latently Perform Multi-Hop Reasoning?. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Lun-Wei Ku, Andre Martins, and Vivek Srikumar (Eds.). Association for Computational Linguistics, Bangkok, Thailand, 10210–10229. <https://doi.org/10.18653/v1/2024.acl-long.550>
- [95] Zairun Yang, Yilin Wang, Zhengyan Shi, Yuan Yao, Lei Liang, Keyan Ding, Emine Yilmaz, Huajun Chen, and Qiang Zhang. 2025. EventRAG: Enhancing LLM Generation with Event Knowledge Graphs. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar (Eds.). Association for Computational Linguistics, Vienna, Austria, 16967–16979. <https://doi.org/10.18653/v1/2025.acl-long.830>
- [96] Jiaxuan You, Jure Leskovec, Kaiming He, and Saining Xie. 2020. Graph structure of neural networks. In *International Conference on Machine Learning*. PMLR, 10881–10891.
- [97] Tian Yu, Hao Zhang, Yichao Liu, Xin Li, Rui Wang, Yifan Li, Yizhou Zhang, Yuxiang Wang, Yuan Chang, and Di He. 2024. Graph Retrieval-Augmented Generation: A Survey. *arXiv preprint arXiv:2405.10515* (2024).
- [98] Chuxu Zhang, Dongjin Song, Chao Huang, Ananthram Swami, and Nitesh V Chawla. 2019. Heterogeneous graph neural network. In *Proceedings of the 25th ACM SIGKDD international conference on knowledge discovery & data mining*. 793–803.
- [99] Michael Zhang and Eunsol Choi. 2023. Mitigating Temporal Misalignment by Discarding Outdated Facts. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, Houda Bouamor, Juan Pino, and Kalika Bali (Eds.). Association for Computational Linguistics, Singapore, 14213–14226. <https://doi.org/10.18653/v1/2023.emnlp-main.879>
- [100] Xinliang Frederick Zhang, Nick Beauchamp, and Lu Wang. 2024. Narrative-of-Thought: Improving Temporal Reasoning of Large Language Models via Recounted Narratives. In *Findings of the Association for Computational Linguistics: EMNLP 2024*, Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen (Eds.). Association for Computational Linguistics, Miami, Florida, USA, 16507–16530. <https://doi.org/10.18653/v1/2024.findings-emnlp.963>
- [101] Zhen Zhang, Zhiyuan Liu, Xu Han, Shuang Li, Peng Li, and Maosong Sun. 2024. Injecting Knowledge Graphs into Large Language Models. *arXiv preprint arXiv:2402.15556* (2024).
- [102] Feng Zhao, Ruoyu Chai, Kangzheng Liu, and Xianggan Liu. 2025. LGA: LLM-GNN Aggregation for Temporal Evolution Attribute Graph Prediction. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, Christos Christodoulopoulos, Tanmoy Chakraborty, Carolyn Rose, and Violet Peng (Eds.). Association for Computational Linguistics, Suzhou, China, 20929–20940. <https://doi.org/10.18653/v1/2025.emnlp-main.1058>
- [103] Yanping Zheng, Lu Yi, and Zhewei Wei. 2024. A survey of dynamic graph neural networks. *Front. Comput. Sci.* 19, 6 (Dec. 2024), 18 pages. <https://doi.org/10.1007/s11704-024-3853-2>
- [104] Jie Zhou, Ganqu Cui, Shengding Hu, Zhengyan Zhang, Cheng Yang, Zhiyuan Liu, Lifeng Wang, Changcheng Li, and Maosong Sun. 2020. Graph neural networks: A review of methods and applications. *AI Open* 1 (2020), 57–81. <https://doi.org/10.1016/j.aiopen.2021.01.001>
- [105] Jun Zhu, Yan Fu, Junlin Zhou, and Duanbing Chen. 2025. A Temporal Knowledge Graph Generation Dataset Supervised Distantly by Large Language Models. *Scientific Data* 12 (05 2025). <https://doi.org/10.1038/s41597-025-05062-0>
- [106] Yinghao Zhu, Changyu Ren, Zixiang Wang, Xiaochen Zheng, Shiyun Xie, Junlan Feng, Xi Zhu, Zhoujun Li, Liantao Ma, and Chengwei Pan. 2024. EMERGE: Enhancing Multimodal Electronic Health Records Predictive Modeling with Retrieval-Augmented Generation. 3549–3559. <https://doi.org/10.1145/3627673.3679582>